jigsaw-agile-community-rules

2nd place solution

Rank	2
Team	^^
Author	Yaoqing Jin
Topic ID	614279
Version	Original

Source: https://www.kaggle.com/competitions/jigsaw-agile-community-rules/discussion/614279
Retrieved with Kaggle CLI on 2026-10-21.

First of all, I want to sincerely thank Kaggle and the Jigsaw team for organizing this very interesting competition, and for opening up a real and high-value problem setting. This task is not just a "classification competition". It is more like being dropped directly into a real-world community moderation environment that is extremely noisy, where the rules can shift, and where even the labels are not always trustworthy.

I would also like to thank my teammates. In areas like solution design, implementation details, debugging, and even mundane work such as notebook version management, my teammates gave me very direct and effective suggestions and sanity checks. They helped me avoid wasting a lot of time. Especially during the final push, when we were discussing how to parallelize training and inference and how to proactively avoid Kaggle's randomness such as "Notebook Threw Exception" or "Time Out", that collaboration was absolutely critical.

I would also like to explain why I am only posting this write-up now. For the past two weeks I have been extremely busy with my actual work. Most of my daytime was spent rushing project deadlines, and I could only clean up code and logs on weekend nights. So this report was delayed until now, Sunday night.

Method Overview

jigsaw-llama3-1-8b-instruct-training-one-epoch was basically the starting point of our approach. Many thanks to @mks2192. It confirmed a very important fact: unsloth is very suitable for doing online LoRA fine-tuning within the Kaggle environment, and it can still maintain very high throughput at inference time. Our later pipeline was essentially an iterative evolution built on top of that.

Our final submission was a multi-model ensemble LLM discriminator system. The core idea was:

1. We treat this task as an instruction tuning problem that looks like "the LLM reads a rule and a comment, and then only answers Yes or No" in a dialogue format.
2. We explicitly perform label cleaning and deduplication on noisy annotations in order to increase the reliability of the supervision signal.
3. We intentionally prevent the model from relying on the subreddit field, because subreddit bias and contamination were extremely severe.
4. We use multiple different base large language models, from different model families and with different sizes, and we fine-tune them with LoRA using unsloth on Kaggle.
5. During inference, we obtain probability scores by inspecting the next-token logit difference between Yes and No.
6. We apply a simple but robust weighted fusion of all model predictions.

---

1. Local Validation Strategy

Early in the competition we found that online learning led to very large fluctuations in the public leaderboard score, and there were many unexpected issues. For example the exact same notebook would sometimes run through and sometimes immediately throw an Exception. This meant that relying only on the five daily submissions made us feel very unsafe.

On the training data we created a StratifiedKFold split with stratification by label and with `val_ratio = 0.1`. In other words we did a stratified split on the binary target `rule_violation`, and we took about 10 percent

of the samples as a local validation set. Every time we changed the method we evaluated it using the average cross-validation score over three random seeds. We computed a local AUC and used this as the offline metric.

This stable local cross-validation score was essentially the anchor for all later decisions.

---

2. Label Clean

The official training data had a practical issue. The same `body` (comment text) paired with the same `rule` (rule text) was sometimes labeled with different `rule_violation` values in different places. In other words the exact same comment and rule pair could sometimes be labeled as violating and sometimes as not violating. If we directly train on these conflicting samples, the LLM will learn that "this question has no standard answer", which makes the supervision signal diverge.

Our cleaning process was:

1. We compress each row into a key. We tried two variants:
   - `"body || rule || subreddit || rule_violation"`
   - and a more aggressive version: `"body || rule || rule_violation"` which means we ignore subreddit
   - In the end our main choice was to ignore subreddit. Section 3 below explains why subreddit is problematic.
2. For the same `(body_trim, rule_trim)` pair, if we observe conflicting labels, we count how many times `rule_violation` is positive versus the total count.
   - We take the majority vote as the final label.
3. We fill the conflicting rows with this majority label.
4. We then deduplicate.

There are two major benefits of this cleaning:

- We get a cleaner supervision signal. The LLM is no longer trained to contradict itself during supervised fine-tuning.
- We get higher quality per sample. After conflict resolution and deduplication, the dataset size actually decreases. However, each remaining sample becomes more trustworthy. For LoRA, which is a low-rank adaptation method, fewer but cleaner samples are more valuable than more but noisy samples, especially because we only have the budget to train for one epoch.

---

3. Intentionally Dropping the Subreddit Feature

We ran a small probe and found that the subreddit field often carries strong social bias signals in the training data. The same rule and the same behavior can be judged as "violation" or "not a violation" depending on the subreddit. The standard, the tone, and the level of tolerance were completely different, and sometimes even chaotic.

For a discriminative LLM, `subreddit` very easily becomes a shortcut feature or even a leakage feature.

In the end we decided on the following main approach. We removed subreddit from the prompt and only kept `Rule:` and `Comment:`.

The prompt basically looked like this:

```
System:
"Decide if the Reddit comment violates the rule. Reply strictly with Yes or No."

User:
Rule: <rule text>
Comment: <comment text>

Assistant:
Yes / No
```

---

4. Multi Model Parallel Fine Tuning and Ensembling

4.1 Model Families

In the end we trained and submitted four LoRA fine-tuned versions, which covered different scales and different LLM architectures:

1. `phi-4-14b` loaded with bnb 4-bit quantization
2. `qwen3-14b` loaded with bnb 4-bit
3. `qwen3-8b` loaded with bnb 4-bit
4. `qwen25-7b-instruct` loaded with bnb 4-bit

All of these models were loaded using `unsloth.FastLanguageModel.from_pretrained(...)` with 4-bit quantized weights, and then we applied LoRA through `LoraConfig`. We used `trl.SFTTrainer` to perform supervised fine tuning for a single epoch.

4.2 Practical Engineering Problems on Kaggle (GPU Parallelism and Rerun Instability)

During the final push we needed to run all four models end to end. This included training, inference, generating `submissionX.csv`, and then fusing the predictions. This was very close to the resource limit on Kaggle. The total runtime of the final submission was 11 hours.

We did two engineering things to make this possible:

(1) Dual GPU Parallel Scheduling We wrote a script called `all_train.sh` and split the models into two groups. We launched them under different `CUDA_VISIBLE_DEVICES` so that two processes could run at the same time.

This approach allowed us to keep the total runtime within a range that Kaggle would still accept.

(2) The Reality That "The Same Notebook Sometimes Works and Sometimes Crashes" We saw a very frustrating Kaggle rerun phenomenon. The exact same notebook version would sometimes finish and produce

a score, and sometimes immediately throw `Notebook Threw Exception` or `Time Out`. Our hypothesis is that Kaggle does not always allocate exactly the same underlying hardware resources for every submission.

Our mitigation strategy was:

- Fix the number of training epochs to 1.
- Choose a batch size that is "high enough but will not cause OOM", for example `per_device_train_batch_size` = 4 or 8, and then use `gradient_accumulation_steps` to control the effective global batch size.
- During inference we used `bs=4` or `bs=per_device_train_batch_size*2`, which are safe values, instead of pushing to extremely large batch sizes.

In other words we intentionally tuned the scripts to use around 80 to 90 percent of the available resources rather than pushing to 100 percent and risking an out of memory error. This made it more likely to survive under a fluctuating environment.

Once again, thanks to Kaggle, the Jigsaw team, and all competitors who discussed and shared baselines. This space of community moderation and content policy judgment is real, difficult, and even a bit chaotic, which is exactly why it is worth continuing to work on.